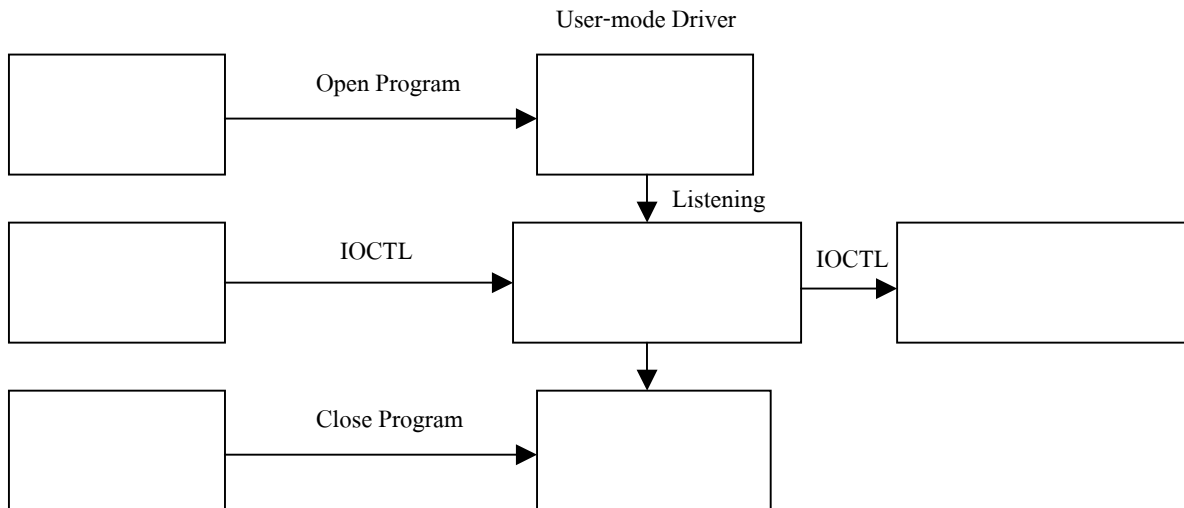


## บทที่ 3

### การออกแบบและการพัฒนา

#### 3.1 การออกแบบ User-mode Driver



รูปที่ 3.1 แสดงการทำงานระหว่าง Application กับ User-mode Driver

การทำงานของ user-mode driver จะทำงาน โดยคอยตอบสนองต่อการทำงานของ Application โดยที่ user-mode driver ส่วนใหญ่จะอยู่ในรูปของไฟล์ DLL มีการทำงานหลักๆ อยู่ 3 ขั้นตอน ดังนี้

1. Create File
2. ทำการรอ เพื่อตอบสนองการทำงานของ Application
3. Close File

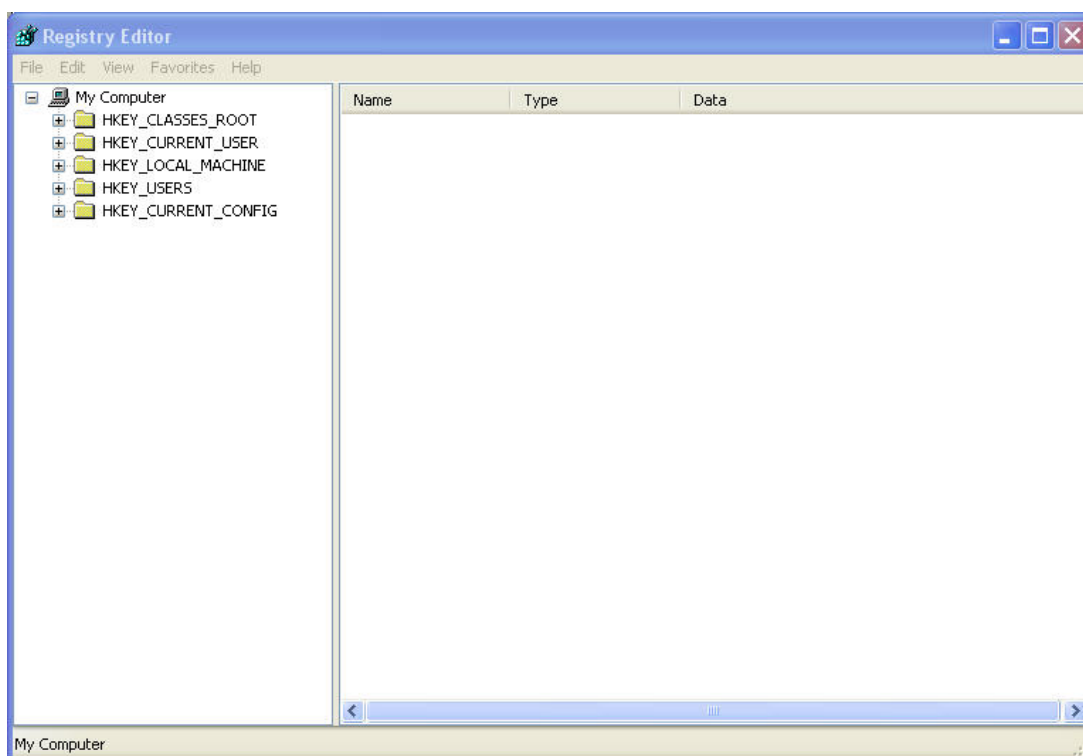
##### 3.1.1 Create File

จะเป็นการสร้างไฟล์ Handle ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อ โดยมีรูปแบบฟังก์ชันดังนี้

```
CreateFile("\\\\.\\Ezusb-0", ..., 0, NULL);
```

ค่าพารามิเตอร์ที่ส่งเข้าไปในฟังก์ชันคือ Device name ในที่นี้คือ Ezusb-0 และพารามิเตอร์ของการ Create File คือ [\\.](#) จะเป็น นามของระบบเช่น C:\windows\system32 ระบบปฏิบัติการจะทำการแปลงเป็น \device\ HarddiskVolume1\Partition0 แต่เราไม่สามารถที่รู้ว่าผู้ใช้ทั่วไปลง

ระบบปฏิบัติการไว้ที่ใด จึงค่าเป็นรูปของระบบซึ่งจะมี Symboliclink ที่จะทำการเชื่อมโยงไปยัง Device name ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อได้ หากเราต้องการตรวจสอบว่า Device name ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อของเราได้ใส่เข้าไปในระบบหรือยัง ก็สามารถทำการตรวจสอบได้โดยการเข้าไปดูที่ Registry ของระบบปฏิบัติการ Windows เข้าไปที่ Run... พิมพ์คำว่า regedit กด OK จากนั้นเราก็จะเข้าสู่หน้าต่าง Registry Editor ของระบบปฏิบัติการ Windows ดังรูปที่ 3.2

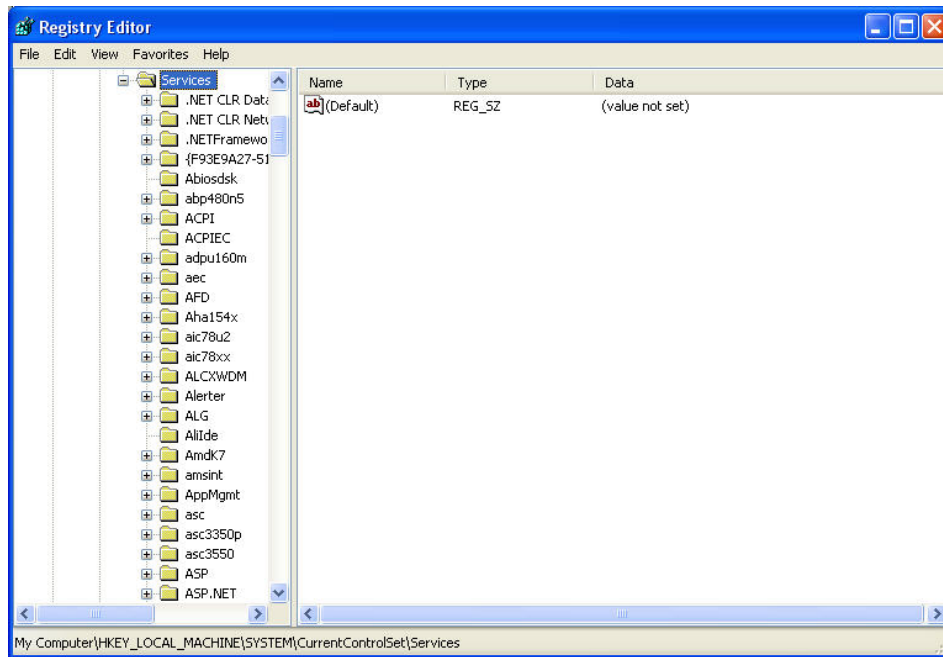


รูปที่ 3.2 หน้าต่าง Registry Editor ของระบบปฏิบัติการ Windows

หากเราต้องการตรวจสอบว่า Device name ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อของเราได้ใส่เข้าไปในระบบหรือยัง ให้เข้าไปดูที่พาธ

RegistryPath = HKEY\_LOCAL\_MACHINE\SYSTEM\CurrentControlSet\Services

ดังรูปที่ 3.3 จะแสดง Device name ของอุปกรณ์ฮาร์ดแวร์ ที่มีอยู่ภายในเครื่อง ที่พาธ HKEY\_LOCAL\_MACHINE\SYSTEM\CurrentControlSet\Services ซึ่งจะมี Device name ของอุปกรณ์ฮาร์ดแวร์หลายชนิด ที่ได้ทำการติดตั้ง Device Driver ลงไปในเครื่องระบบปฏิบัติการ Windows



รูปที่ 3.3 พาท HKEY\_LOCAL\_MACHINE\SYSTEM\CurrentControlSet\Services

โดยรูปแบบของโปรแกรมที่เขียนขึ้นมามีดังนี้

```
char completeDeviceName[64] = "";
char pcMsg[64] = "";
strcat (completeDeviceName, "\\.\\" );
strcat (completeDeviceName, " Ezusb-0");
*phDeviceHandle = CreateFile( completeDeviceName,
                             GENERIC_WRITE,
                             FILE_SHARE_WRITE,
                             NULL,
                             OPEN_EXISTING, 0,
                             NULL);

if (*phDeviceHandle == INVALID_HANDLE_VALUE) {
    return (FALSE);
} else {
    return (TRUE);
}
```

สิ่งที่ได้จากการ Create File คือ Handle ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อกับ

### 3.1.2 ทำการรอ เพื่อตอบสนองการทำงานของ Application

user-mode driver เมื่อทำการ Create File และได้ Handle ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อเรียบร้อยแล้ว ก็จะทำการรอคอยเพื่อตอบสนองการทำงานต่อ Application เมื่อ Application ต้องการที่จะเขียนข้อมูลไปยังอุปกรณ์ฮาร์ดแวร์ ก็จะส่ง IoControlCode ไปให้กับ user-mode driver และ user-mode driver ก็จะทำการส่งข้อมูลที่จะเขียนไปยัง kernel-mode driver

ในการส่งข้อมูล user-mode driver จะส่งข้อมูลไปเป็นแพ็คเกจ โดย 1 แพ็คเกจจะส่งคำสั่งไปจำนวน 3 ชุด ฟังก์ชันที่ใช้ในการส่งข้อมูลมีรูปแบบดังนี้

```
DeviceIoControl(Handle, Code, InputData, InputLength,
                OutputData, OutputLength);
```

|              |                                              |
|--------------|----------------------------------------------|
| Handle       | Handle ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อ |
| Code         | IoControlCode                                |
| InputData    | ข้อมูลที่ต้องการจะเขียน                      |
| InputLength  | ขนาดของข้อมูลที่จะเขียน                      |
| OutputData   | ข้อมูลที่ถูกส่งกลับมา                        |
| OutputLength | ขนาดของข้อมูลที่ส่งกลับมา                    |

ค่าที่ได้รับกลับมาจากฟังก์ชันนี้คือ BOOL

ถ้าเป็น TRUE แสดงว่าการส่งข้อมูลเป็นผลสำเร็จ

ถ้าเป็น FALSE แสดงว่าการส่งข้อมูลเป็นไม่ผลสำเร็จ

ตัวอย่าง IOCTL (I/O Control Code) ที่ส่งออกไปให้กับ kernel-mode driver

•IOCTL\_EZUSB\_BULK\_WRITE

•IOCTL\_EZUSB\_BULK\_READ

สำหรับการเขียนข้อมูล 1 ครั้ง จะส่งข้อมูลไป 3 ชุด

ชุดที่1 จะเป็นการส่ง command ออกไปให้กับอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อโดยจะส่งข้อมูลออกไปทาง PIPE 6 เพื่อเป็นการเช็คค่าให้กับอุปกรณ์ฮาร์ดแวร์

ชุดที่2 จะเป็นการส่งข้อมูล ออกไปให้กับอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อโดยจะส่งข้อมูลออกไปทาง PIPE 4 เพื่อเป็นการบอกว่าจะเขียนข้อมูลออกไปที่ address ที่เท่าไร

ชุดที่3 จะเป็นการส่งข้อมูล ออกไปให้กับอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อโดยจะส่งข้อมูลออกไปทาง PIPE 4 เพื่อเป็นการบอกว่าข้อมูลที่ต้องการจะเขียนคืออะไร

โดยรูปแบบของโปรแกรมที่เขียนขึ้นมีดังนี้

```

bulkControl.pipeNum = 6;
if (bOpenDriver (&hDevice, pcDriverName) != TRUE){
    hDevice = NULL;
}
if (hDevice == NULL){
    m_output += "PC didn't connect board\x0d\x0a";
}
if (hDevice != NULL){
    bResult = DeviceIoControl(hDevice,
                               ioctl_val,
                               &bulkControl,
                               sizeof (BULK_TRANSFER_CONTROL),
                               command,
                               5,
                               (unsigned long *)&nBytes,
                               NULL);

    bulkControl.pipeNum = 4;
    bResult = DeviceIoControl(hDevice,
                               ioctl_val,
                               &bulkControl,
                               sizeof (BULK_TRANSFER_CONTROL),
                               command,
                               5,
                               (unsigned long *)&nBytes,
                               NULL);

    bulkControl.pipeNum = 2;
    bResult = DeviceIoControl(hDevice,
                               ioctl_val,
                               &bulkControl,
                               sizeof (BULK_TRANSFER_CONTROL),
                               &data,

```

```
length,
(unsigned long *)&nBytes,
NULL);
```

จากโปรแกรมที่เขียนขึ้นมา มีการใช้ฟังก์ชัน DeviceIoControl ส่งข้อมูลออกไปเป็นจำนวน 3 ครั้ง โดยครั้งแรกเป็นการส่งออกไปที่ไปป์หมายเลข 6 เพื่อส่ง command ออกไป และค่าที่อยู่ในตัวแปร command คือค่าคำสั่งและค่า address เริ่มต้นที่ต้องการจะส่งข้อมูลออกไป โดยการส่งครั้งแรกออกไปที่ไปป์หมายเลข 6 เพื่อเป็นการตั้งค่าให้กับอุปกรณ์ฮาร์ดแวร์ว่าต้องการทำงานที่โหมดใด ถ้าไม่มีการตั้งค่านี้อาจส่งข้อมูลออกไป จะเป็นการเขียนข้อมูลออกไปที่ address เดิมบวกกับจำนวนขนาดของข้อมูล ดังนั้นค่า address จะบวกเพิ่มขึ้นทุกๆ ครั้งที่มีการส่งข้อมูล ส่วนการส่งครั้งที่ 2 ส่งออกไปที่ไปป์หมายเลข 4 เป็นการตั้งค่า address เริ่มต้นที่จะส่งข้อมูลออกไป ส่วนการส่งครั้งสุดท้าย จึงจะเป็นข้อมูลจริงที่ต้องการจะส่งออกไป จากโค้ดข้างบนจะมีส่วนตรวจเช็คว่าคุณสมบัติได้ต่ออยู่กับเครื่องหรือไม่ ถ้าไม่ได้ต่ออยู่จะทำให้ไม่สามารถสร้างไฟล์ Handle ได้ ดังนั้นถ้าอุปกรณ์ไม่ได้ต่ออยู่ก็จะให้แสดงข้อความว่า “PC didn't connect board”

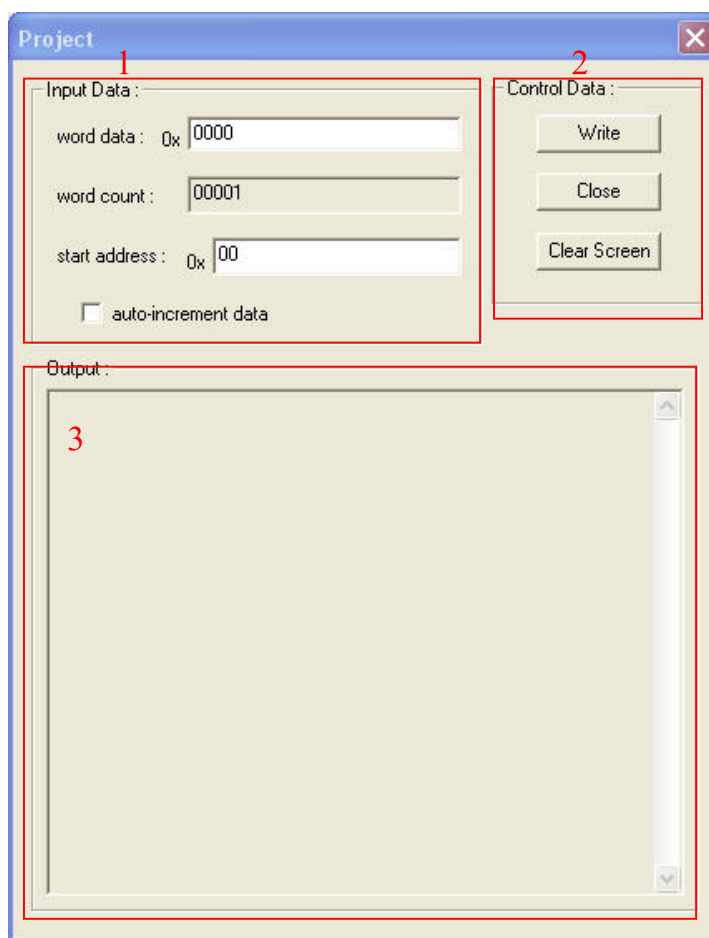
### 3.1.3 Close File

จะเป็นการปิดการเชื่อมต่อกับอุปกรณ์ฮาร์ดแวร์โดยมีรูปแบบฟังก์ชันดังนี้

```
CloseHandle(Handle);
```

ค่าพารามิเตอร์ที่ส่งเข้าไปในฟังก์ชันคือ Handle ของอุปกรณ์ฮาร์ดแวร์ที่ได้ทำการ Create File ได้ Create File เอาไว้ในขั้นตอนที่ 1

### 3.2 การออกแบบโปรแกรมที่ทำงานติดต่อกับ User-mode Driver



รูปที่ 3.4 User interface ของโปรแกรม

#### 1. ส่วนที่ 1

1. Edit box “word data”: ใส่ข้อมูลที่ต้องการจะเขียนไปยัง board
2. Edit box “word count”: ใส่จำนวนครั้งที่ต้องการจะส่งข้อมูลออกไป
3. Edit box “start address”: ใส่ address เริ่มต้นของการส่งข้อมูล
4. Radio box “auto-increment address”: เลือกถ้าผู้ใช้ต้องการให้เพิ่ม address อัตโนมัติเมื่อทำการอ่านเขียน

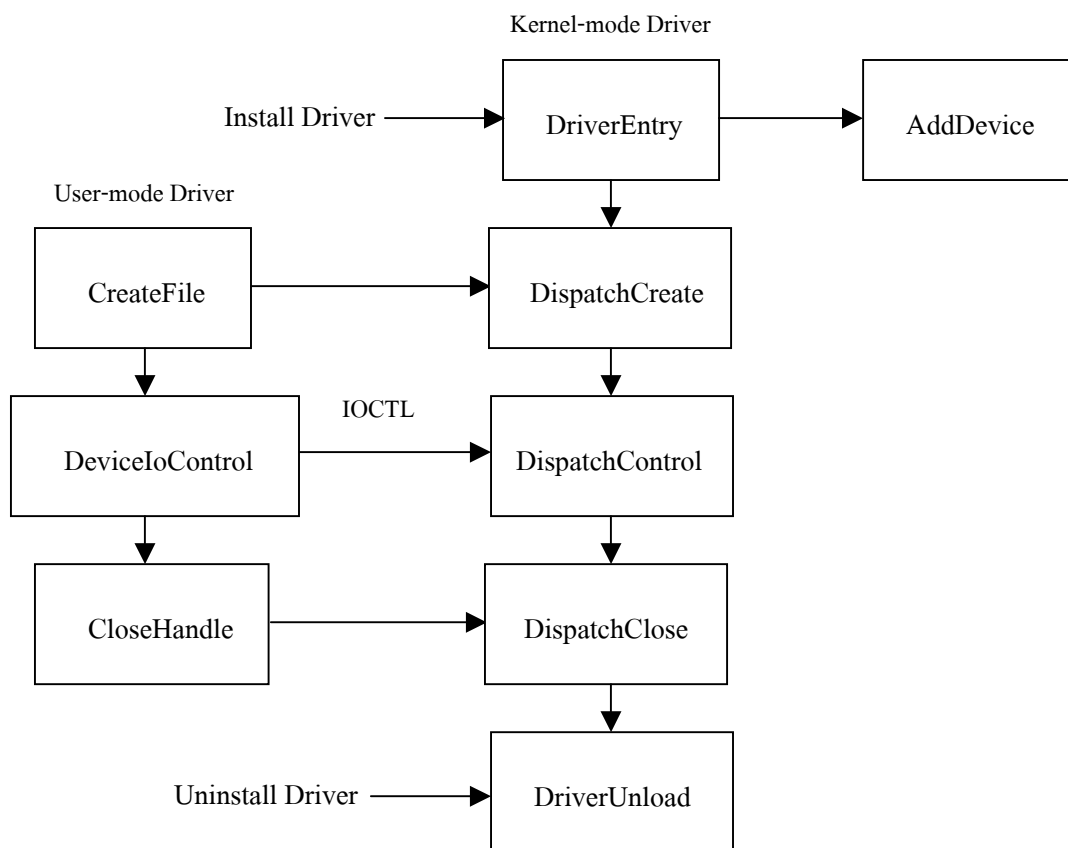
#### 2. ส่วนที่ 2

1. Button “Write”: คลิกเพื่อเขียนข้อมูลไปยัง board
2. Button “Close”: คลิกเพื่อปิดโปรแกรม
3. Button “Clear Screen”: คลิกเพื่อลบข้อมูลทั้งหมดใน edit box

### 3. ส่วนที่ 3

1. Edit box ที่ใหญ่ที่สุด: รายงานข้อมูลที่ถูกเขียนและ address ที่ถูกเขียน

### 3.3 การออกแบบ Kernel-mode Driver



รูปที่ 3.5 แสดงการทำงานระหว่าง User-mode Driver กับ kernel-mode Driver

การทำงานของ kernel-mode driver จะทำงานโดยคอยตอบสนองต่อการทำงานของ user-mode driver โดยที่ kernel-mode driver จะอยู่ในรูปของไฟล์ SYSTEM (\*.sys) มีการทำงานของฟังก์ชันหลักๆ อยู่ 6 ฟังก์ชัน ดังนี้

1. DriverEntry
2. AddDevice
3. DispatchCreate
4. DispatchControl
5. DispatchClose
6. DriverUnload



### 3.3.1 DriverEntry

เป็นฟังก์ชันของ kernel-mode driver เพราะเป็นจุดทางเข้าไปใน kernel-mode driver การทำงานส่วนใหญ่จะเป็นการกำหนดค่าเริ่มต้นให้กับ driver คือเมื่อมี IRP อะไรเข้ามาจะให้ไปทำงานที่ฟังก์ชันไหน มีรูปแบบของฟังก์ชันดังนี้

```
DriverEntry(IN PDRIVER_OBJECT DriverObject, IN PUNICODE_STRING
RegistryPath)
```

ค่าพารามิเตอร์ตัวแรกที่เข้ามา คือ DriverObject เป็นตัวแปร Struct ที่มี data member เป็น pointer to function เป็นการกำหนดค่าเริ่มต้นเพื่อบอกให้ driver รู้ว่าให้ไปทำงานต่อที่ฟังก์ชันอะไร เช่น

```
DriverObject->DriverUnload = DriverUnload;
DriverObject->DriverExtension->AddDevice = AddDevice;
DriverObject->MajorFunction[IRP_MJ_CREATE] = DispatchCreate;
DriverObject->MajorFunction[IRP_MJ_CLOSE] = DispatchClose;
DriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL] = DispatchControl;
```

ค่าพารามิเตอร์ตัวที่ 2 คือ RegistryPath เป็นตัวแปรที่เก็บค่าของ registry path ที่ driver จะทำการใส่ device name ลงไปในระบบ ซึ่งค่า RegistryPath จะมีค่าดังนี้

```
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services
```

### 3.3.2 AddDevice

เป็นฟังก์ชันที่ใช้ในการกำหนดค่าเริ่มต้นและทำการสร้าง DeviceObject ซึ่งเป็นตัวแปรที่เก็บค่า IRP และค่าต่างๆ ของ driver มีรูปแบบของฟังก์ชันดังนี้

```
AddDevice(PDRIVER_OBJECT DriverObject, PDEVICE_OBJECT deviceObject )
```

ค่าพารามิเตอร์ตัวแรกที่เข้ามา คือ DriverObject เป็นตัวแปร Struct ตัวเดียวกันกับที่อยู่ใน DriverEntry ที่มี data member เป็น pointer to function เป็นการกำหนดค่าเริ่มต้นเพื่อบอกให้ driver รู้ว่าให้ไปทำงานต่อที่ฟังก์ชันอะไร

ค่าพารามิเตอร์ตัวที่ 2 คือ DeviceObject เป็นตัวแปร Struct ที่มี data member เป็นค่าการทำงานต่างๆ ที่ได้รับเข้ามาจากระบบปฏิบัติการและเป็นค่าที่ได้รับเข้ามาจาก user-mode driver การทำงานหลักๆ ของฟังก์ชันนี้คือ

1. ทำการสร้าง device name โดยใช้ฟังก์ชัน IoCreateDevice มีรูปแบบดังนี้

```
NTSTATUS IoCreateDevice (DriverObject, sizeof (DEVICE_EXTENSION),
                        &deviceNameUnicodeString,
                        FILE_DEVICE_UNKNOWN, 0, FALSE,
                        &deviceObject);
```

2. ทำการสร้าง Symboliclink โดยการใช้ฟังก์ชัน IoCreateSymbolicLink มีรูปแบบดังนี้

```
NTSTATUS IoCreateSymbolicLink (&deviceLinkUnicodeString,
                              &deviceNameUnicodeString);
```

### 3.3.3 DispatchCreate

เป็นฟังก์ชันที่ตอบสนองต่อการทำงานของ user-mode driver เมื่อมีการใช้ฟังก์ชัน CreateFile ที่โปรแกรมบน user-mode driver ก็จะมีการทำงานบน kernel-mode driver ที่ฟังก์ชัน DispatchCreate มีรูปแบบของฟังก์ชันดังนี้

```
NTSTATUS DispatchCreate(PDEVICE_OBJECT deviceObject, PIRP Irp)
```

การทำงานของฟังก์ชันนี้จะมีการเรียกใช้ ฟังก์ชัน IoGetCurrentIrpStackLocation(Irp) เพื่อนำค่าของ stack ออกมา เพื่อนำมาหา IRP ที่ได้รับเข้ามาเพื่อตอบสนองต่อการทำงาน และจะมีการเรียกใช้ฟังก์ชัน InterlockedIncrement(&pdx->handles) เพื่อทำการล็อกการทำงานเพื่อใช้ในการทำงานแบบ Synchronization เพราะการทำงานของ driver จะทำงานทีละ request เพราะฉะนั้นจึงต้องมีการล็อก เพื่อให้ request อื่นมาแทรกการทำงาน

### 3.3.4 DispatchControl

เป็นฟังก์ชันที่ตอบสนองต่อการทำงานของ user-mode driver เมื่อมีการใช้ฟังก์ชัน DeviceIoControl ที่โปรแกรมบน user-mode driver ก็จะมีการทำงานบน kernel-mode driver ที่ฟังก์ชัน DispatchControl มีรูปแบบของฟังก์ชันดังนี้

```
NTSTATUS DispatchControl(PDEVICE_OBJECT fdo, PIRP Irp)
```

การทำงานก็จะทำงานตอบสนองต่อ IOCTL ที่ได้รับเข้ามาจาก user-mode driver และนำมาเปรียบเทียบกับ IOCTL และทำงานตาม IOCTL ที่ได้รับเข้ามา การทำงานจะมีดังนี้

```

PIO_STACK_LOCATION stack = IoGetCurrentIrpStackLocation(Irp);
ULONG cbin = stack->Parameters.DeviceIoControl.InputBufferLength;
ULONG cbout = stack->Parameters.DeviceIoControl.OutputBufferLength
ULONG code = stack->Parameters.DeviceIoControl.IoControlCode;

switch (code){
    case IOCTL_EZUSB_BULK_WRITE :
    ...
}

```

เมื่อได้รับ IOCTL เข้ามาจาก user-mode driver ก็จะนำมาคำค่าของ InputBuffer จากคำสั่ง stack->Parameters.DeviceIoControl.InputBufferLength และหาคำค่าของ OutputBuffer จากคำสั่ง stack->Parameters.DeviceIoControl.OutputBufferLength และหาคำค่าของ IOCTL จากคำสั่ง stack->Parameters.DeviceIoControl.IoControlCode เมื่อได้ IOCTL แล้วก็จะนำมาเปรียบเทียบกับเพื่อทำงานต่อไป

### 3.3.5 DispatchClose

เป็นฟังก์ชันที่ตอบสนองต่อการทำงานของ user-mode driver เมื่อมีการใช้ฟังก์ชัน CloseFile ที่โปรแกรมบน user-mode driver ก็จะมีการทำงานบน kernel-mode driver ที่ฟังก์ชัน DispatchCreate มีรูปแบบของฟังก์ชันดังนี้

```
NTSTATUS DispatchClose(PDEVICE_OBJECT deviceObject, PIRP Irp)
```

การทำงานของฟังก์ชันนี้จะมีการเรียกใช้ ฟังก์ชัน IoGetCurrentIrpStackLocation(Irp) เพื่อนำค่าของ stack ออกมา เพื่อนำมาหา IRP ที่ได้รับเข้ามาเพื่อตอบสนองต่อการทำงาน และจะมีการเรียกใช้ฟังก์ชัน InterlockedDecrement(&pdx->handles) เพื่อทำการปลดล็อกการทำงานเพื่อที่ใช้ในการทำงานแบบ Synchronization เพราะเมื่อมีการทำงานเสร็จแล้วจะต้องทำการปลดล็อกเพื่อให้ request อื่นเข้ามาทำงานต่อไป

### 3.3.6 DriverUnload

จะทำงานก็ต่อเมื่อมีการ Uninstall driver การทำงานจะเป็น Deallocate ตัวแปรต่างๆ ที่ได้ทำการสร้างขึ้นมาใช้ มีรูปแบบดังนี้

```
VOID DriverUnload(IN PDRIVER_OBJECT DriverObject)
```

ในการเขียน driver ทั่วไป จะมีการใช้ตัวแปรชนิด UnicodeString ซึ่งจะต้องมีการสร้างตัวแปรชนิดนี้ขึ้นมา เช่น ใช้ในการกำหนด device name ก็จะใช้ตัวแปรแบบ UnicodeString เพราะฉะนั้น เมื่อทำการ Uninstall driver จะต้องใช้คำสั่ง RtlFreeUnicodeString(&servkey) ในการทำลายตัวแปรนี้

### 3.4 การออกแบบไฟล์ INF

เป็นไฟล์ที่ใช้ในการ install device driver ซึ่งรายละเอียดภายในไฟล์นี้จะมีการระบุข้อมูลต่างๆของอุปกรณ์ฮาร์ดแวร์ และมีรหัสของอุปกรณ์ด้วย โดยอุปกรณ์ที่ใช้เป็นอุปกรณ์ USB มีการกำหนดรายละเอียดมีดังนี้

```
[Version]
```

```
..
```

```
Class=USB
```

```
..
```

```
[DestinationDirs]
```

```
DefaultDestDir=10,System32\Drivers
```

```
[SourceDisksFiles]
```

```
test.sys=1
```

```
[SourceDisksNames]
```

```
1=%INSTDISK%,,,,objchk\i386
```

```
[DeviceList]
```

```
%DESCRIPTION%=DriverInstall,
```

```
USB\VID_ffff&PID_ddd&REV_rrrr[DriverInstall.NT]
```

```
CopyFiles=DriverCopyFiles
```

```
[DriverCopyFiles]
```

```
test.sys,,,
```

```
test.jnk,,,
```

[Strings]

INSTDISK="Project Disc"

DESCRIPTION="Test Project"

ส่วน [Version] กำหนด Class เป็น USB

ส่วน [DestinationDirs] กำหนดพาธไปที่ C:\WINDOWS\system32\drivers และใช้ตัวแปรระบบ 10 เพื่อในกรณีที่ผู้ใช้ไม่ได้ลงระบบปฏิบัติการ windows ไว้ที่ไดรว์ C กำหนดพาธเพื่อใช้ในกรณีที่มีการคัดลอกไฟล์ระบบของอุปกรณ์ฮาร์ดแวร์ จะได้ทำการคัดลอกไปไว้ที่พาธ C:\WINDOWS\system32\drivers เพื่อนำไปใช้กับ driver และระบบปฏิบัติการ windows

ส่วน [SourceDisksFiles] กำหนดว่า ไฟล์ไดรว์เวอร์ชื่อ test.sys

ส่วน [SourceDisksNames] กำหนดว่าไฟล์ไดรว์เวอร์อยู่พาธ objchki386 ของดิสก์

ส่วน [DeviceList] กำหนดรหัสของอุปกรณ์ และมีการกำหนดให้มีการคัดลอกไฟล์ไปไว้ที่พาธ C:\WINDOWS\system32\drivers ด้วย โดยในโครงงานนี้เป็นอุปกรณ์ USB มีการกำหนดค่าดังนี้

```
USB\VID_vvvv&PID_dddd&REV_rrrr
```

vvvv คือ เลขฐาน 16 จำนวน 4 บิต เป็นรหัสผู้ผลิต

dddd คือ เลขฐาน 16 จำนวน 4 บิต เป็นรหัสของอุปกรณ์

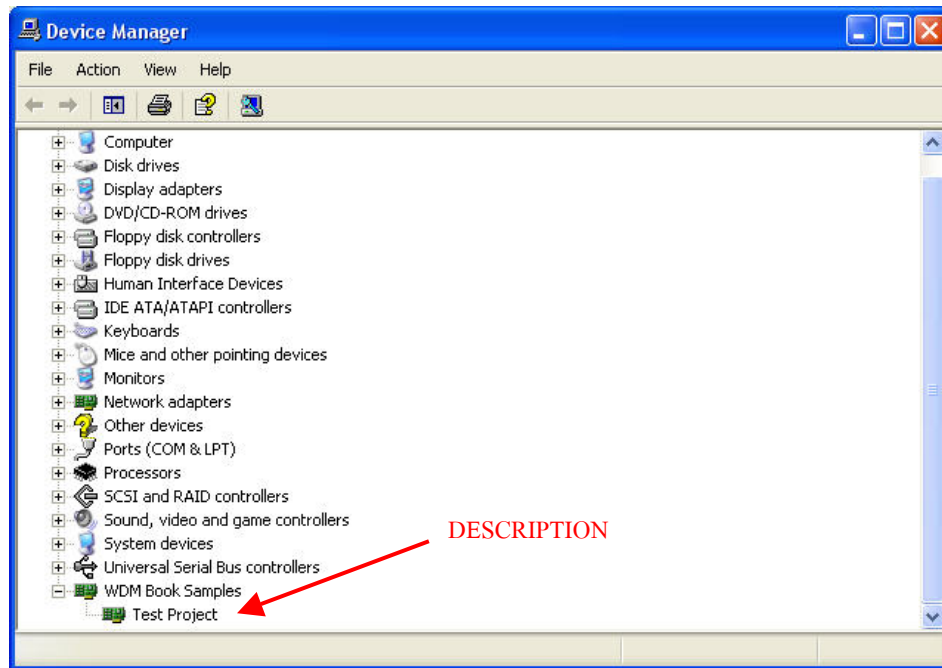
rrrr คือ รหัสการแก้ไขปรับปรุง

ส่วน [DriverCopyFiles] เป็นการกำหนดให้การ install device driver ให้ทำการคัดลอกไฟล์ 2 ไฟล์คือ test.sys และ test.jnk ซึ่ง test.sys เป็นไฟล์ไดรว์เวอร์ และ test.jnk เป็นไฟล์ที่ใช้ทำงานร่วมกับไดรว์เวอร์ไปไว้ที่พาธ C:\WINDOWS\system32\drivers

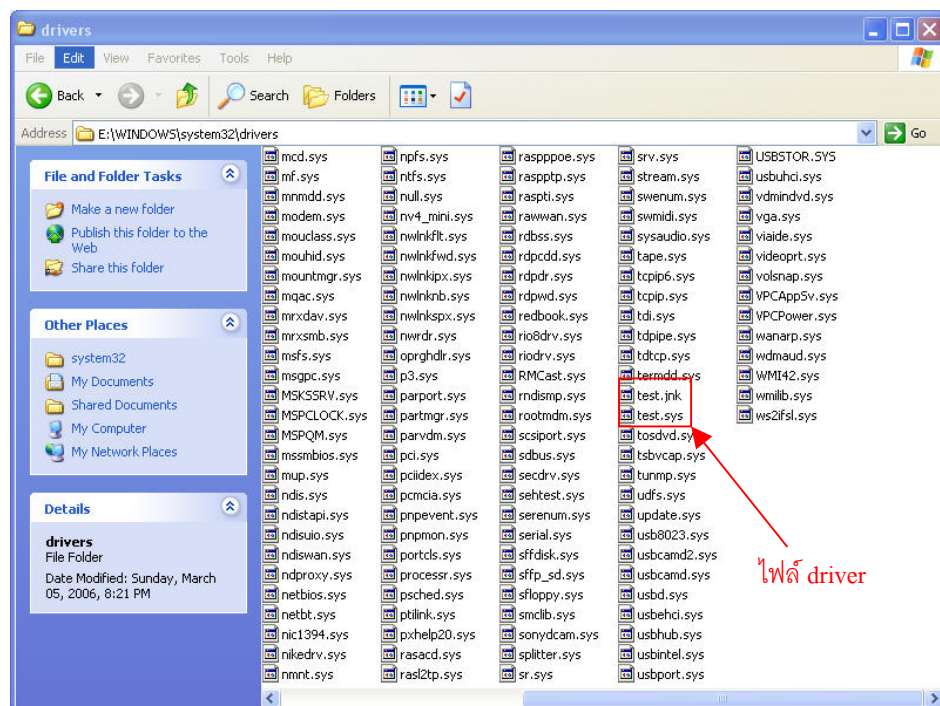
ส่วน [Strings] เป็นการกำหนดค่าตัวแปร string ที่ใช้ภายในไฟล์ INF โดยมีการกำหนดไว้ 2 ตัวแปรคือ INSTDISK และ DESCRIPTION ซึ่งแต่ละตัวแปรมีรายละเอียดดังนี้

INSTDISK คือ ชื่อของดิสก์ที่เก็บไฟล์ driver เอาไว้

DESCRIPTION คือ ชื่อที่ใช้แสดงในหน้าต่าง Device Manager เมื่อทำการ install device driver เสร็จแล้วดังแสดงในรูปที่ 3.6



รูปที่ 3.6 หน้าต่าง Device Manager



รูปที่ 3.7 C:\WINDOWS\system32\drivers

จากรูปที่ 3.7 แสดงพบ C:\WINDOWS\system32\drivers ที่เก็บไฟล์ test.sys และ test.jnk ซึ่งเป็นไฟล์ไดรเวอร์และใช้ที่ทำงานร่วมกับไฟล์ไดรเวอร์

### 3.5 ขั้นตอนในการออกแบบและพัฒนา USB

ในโปรเจกต์ทั่วไป ถ้าคุณต้องการที่จะสร้างโปรเจกต์ขึ้นมา เราต้องทำการแบ่งงานออกเป็นส่วนๆก่อน ที่จะทำให้การลงมือทำ ซึ่งในการเขียนโปรแกรมภายในตัวชิปนั้น คุณสามารถเริ่มได้โดยการเขียนโค้ดให้ระบบปฏิบัติการนั้นให้พอที่จะหาตัวอุปกรณ์นั้น พบและสามารถทำการระบุไปที่อุปกรณ์ที่เราได้เขียนขึ้นมา จากนั้นคุณค่อยมาทำการแลกเปลี่ยนข้อมูลอย่างง่าย ๆ กับตัวโปรแกรมที่รันอยู่บนระบบปฏิบัติการ จากนั้นคุณสามารถที่จะมาเพิ่มส่วนที่เป็นรายละเอียดปลีกย่อยที่จะทำให้สามารถติดต่อกับโปรแกรมที่รันอยู่บนระบบปฏิบัติการได้ ซึ่งขั้นตอนในการพัฒนาโปรเจกต์นั้น ก็จะประกอบด้วย การ initial decisions ,enumerating และ exchanging data

#### 3.5.1 Initial decisions

ก่อนที่จะเริ่มพัฒนาโปรเจกต์ได้นั้น คุณจำเป็นต้องทำการรวบรวมข้อมูลและต้องทำบางอย่างในการที่จะใช้ในการตัดสินใจกับตัวโปรเจกต์ของเรา

1. อย่างแรกคุณต้องระบุรายละเอียดตัวอุปกรณ์ของคุณสำหรับในการติดต่อกับตัวUSB ของคุณ ว่าคุณต้องการที่จะรับส่งข้อมูลที่มีขนาดมากเท่าไรและความเร็วในการรับส่งข้อมูลในตัวอุปกรณ์ที่คุณทำขึ้นมาขึ้นมานั้นต้องการในการทำงาน
2. นำเอาคำตอบในข้อที่ 1 มาทำการระบุความต้องการของตัวชิปของเรา ถ้าคุณมีชิปที่เอาไว้อ้างอิงอยู่แล้วก็ให้กำหนดมันลงไป
3. นำเอาความต้องการของคุณมาใช้ในการตัดสินใจว่า คอมพิวเตอร์ของคุณจะติดต่อในการส่งข้อมูลกับตัวอุปกรณ์ของคุณโดยใช้ตัว device driver ทั่วไป หรือจะเป็นตัว device driver ที่คุณทำขึ้นมา
4. เลือก controller chip ที่ตรงกับความต้องการของคุณ

#### 3.5.2 Enumerating

ในส่วนนี้ให้เราเอาอะไรก็ตามที่คุณจำเป็นต้องเอาหรือรับจากระบบปฏิบัติการมาทำการระบุตัวอุปกรณ์ของคุณ

1. การเขียนโค้ดควบคุมชิปตามรายการในการระบุตัวอุปกรณ์ ซึ่งรายละเอียดจะเปลี่ยนแปลงตามชิป ซึ่งชิปนั้นต้องสามารถติดต่อกับdescriptors ซึ่งชิปนั้นต้องมีโปรแกรม หรืออุปกรณ์ที่สามารถจำและตอบสนองความต้องการของโค้ดในโปรแกรมที่โอสต์ส่งมาเมื่อมันทำการระบุตัวอุปกรณ์ ซึ่งผู้ขายชิปทั่วไปจะจัดเตรียมโค้ดตัวอย่างซึ่งคุณสามารถใช้กับการปรับปรุงเล็กน้อยๆ
2. สร้างหรือเอา INF ไฟล์ซึ่งระบบปฏิบัติการสามารถทำการชี้ไปที่อุปกรณ์ของคุณเมื่อทำการระบุมัน ซึ่งINF file name คือชื่อที่อุปกรณ์เรานั้นใช้ในส่วนนี้ คุณสามารถใช้ driver

ทุกๆ ไปที่สามารถสนับสนุนชิปของคุณซึ่งผู้ขายส่วนมากจะจัดเตรียมตัวอย่าง INF file และ driver ไว้ให้คุณ

3. ถ้าจำเป็นต้องออกแบบและสร้างวงจรที่จำกัดต่อกับชิปไปที่โฮสต์ซึ่งส่วนใหญ่ในกรณีที่ คุณทำการใส่ค่าเริ่มต้นในการใช้พัฒนานาบอร์ด
4. โหลดโค้ดของคุณไปสู่อุปกรณ์ของคุณและเสียบอุปกรณ์ของคุณใส่ Host และระบบปฏิบัติการควรจะทำภาระบนตัวอุปกรณ์ และเพิ่มมันลงใน control panel และชี้ไปยังตัวที่ ถูกต้อง

### 3.5.3 Exchanging Data

ขั้นตอนเหล่านี้มีความสัมพันธ์กับfunction ที่จะกระทำกับอุปกรณ์

1. เพิ่มความสามารถที่ตัวอุปกรณ์โดยเพิ่มโค้ดที่ตัวโปรแกรมภายในชิปและส่วนต่างๆ ที่ติดต่อกับชิป
2. ถ้าคุณใช้custom driver คุณต้องเขียนdriver ในการติดต่อกับอุปกรณ์
3. ถ้าจำเป็นต้องเขียนโปรแกรมในการติดต่อกับอุปกรณ์ ซึ่งถ้าคุณกำลังออกแบบ mouse ,keyboard หรือ อุปกรณ์ทั่วไป คุณสามารถที่จะติดต่อกับอุปกรณ์ได้โดยโปรแกรมใดๆ
4. ทำการdebug และ วนซ้ำถ้าจำเป็น
5. เมื่อโค้ดถูกdebug แล้วคุณก็พร้อมที่จะโหลดโปรแกรมลงในชิป และให้ทำการทดสอบตัว อุปกรณ์รอบสุดท้าย

แต่ก่อนที่คุณเริ่มในส่วนนี้ มันต้องใช้ประโยชน์ในส่วนที่คุณรู้ในเรื่องเกี่ยวกับการระบุ อุปกรณ์ และการรับส่งข้อมูลกับอุปกรณ์ ดังนั้นคุณสามารถไปใช้ในการเลือกชิปและdriverของคุณได้

### 3.5.4 Tools

ในการพัฒนาอุปกรณ์เกี่ยวกับ USB นั้น คุณจำเป็นต้องมี tool ที่จะนำมาช่วยเราในการพัฒนาโปรแกรมที่เราใช้ดังนี้

1. assembler หรือ compiler ที่สามารถสร้าง firmware(โค้ดที่รันอยู่ในcontroller chip) ได้ ถ้าคุณใช้โค้ด assembly คุณจำเป็นต้องทำการแปลงโค้ดที่เป็นภาษาassemblyของคุณให้เป็น machine code ให้ controller เข้าใจ แต่ถ้าคุณใช้ภาษา ซี หรือ ภาษาอื่น คุณจำเป็นต้องหาตัวที่สามารถทำการแปลงโค้ดให้มาเป็น machine code ให้ได้เพื่อจะนำมาใช้ในการควบคุมของคุณ
2. โปรแกรมเมอร์จะเก็บโค้ดที่ทำเสร็จแล้วไว้ใน controller program memory



3. โสสต์ อาจประกอบด้วย device driver หรือ mini-driver หรือ application code ในการสร้าง device driver คุณจำเป็นต้องใช้ Visual C++ ที่ซึ่งสามารถทำ WDM(Win32 Driver Model) ซึ่งนำมาใช้กับ USB ได้
4. โปรแกรมที่ใช้ในการดักจับเหตุการณ์ที่เกิดขึ้นภายในโปรแกรม หรือ เครื่องมือที่ใช้ในการ debug อื่นๆสำหรับการพัฒนาโปรแกรม

### 3.6 อุปกรณ์ฮาร์ดแวร์

อุปกรณ์ฮาร์ดแวร์ชิ้นนี้เป็นผลิตภัณฑ์ของบริษัท Design Gateway Company Limited เป็นระบบที่สามารถใช้เชื่อมต่อคอมพิวเตอร์กับฮาร์ดแวร์ที่พัฒนาขึ้นเอง ทำให้ฮาร์ดแวร์นี้สามารถติดต่อกับพอร์ต USB ได้โดยที่ผู้ใช้ไม่ต้องมีความรู้เกี่ยวกับพอร์ต USB ทำให้ผู้ใช้สามารถพัฒนาผลิตภัณฑ์ได้เร็วขึ้น นอกจากนั้น timing diagram ของอุปกรณ์ฮาร์ดแวร์ชิ้นนี้ ยังเหมือนกับของหน่วยประมวลผล Z80 อีกด้วย

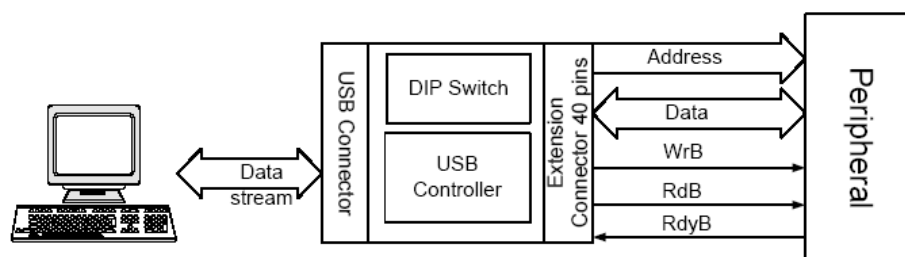
#### 3.6.1 คุณลักษณะของอุปกรณ์ฮาร์ดแวร์

1. อุปกรณ์ฮาร์ดแวร์ชิ้นนี้เชื่อมต่อกับคอมพิวเตอร์ด้วยพอร์ต USB 1.1
2. อุปกรณ์ฮาร์ดแวร์ชิ้นนี้เชื่อมต่อกับฮาร์ดแวร์ที่ต้องการได้ด้วย connector 40 pin ซึ่งประกอบด้วย address bus 8 บิต, data bus 16 บิต, สัญญาณ read (RdB) 1 บิต, สัญญาณ write (WrB) 1 บิต, สัญญาณ ready (RdyB) 1 บิต, ground, และ Vcc

#### 3.6.2 Block Diagram ของอุปกรณ์ฮาร์ดแวร์

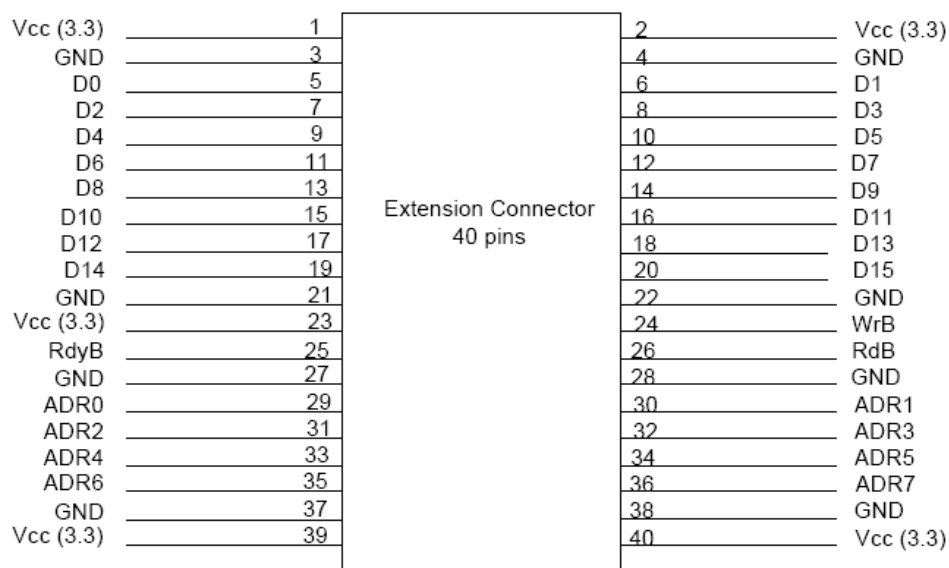
Block diagram ของอุปกรณ์ฮาร์ดแวร์ชิ้นนี้ประกอบด้วย

1. ชิพ USB controller ซึ่งทำหน้าที่ติดต่อระหว่างคอมพิวเตอร์และอุปกรณ์ที่ต้องการ
2. USB connector ถูกใช้เพื่อเสียบสาย USB ระหว่างคอมพิวเตอร์และวงจรของอุปกรณ์ฮาร์ดแวร์
3. DIP switch ถูกใช้เพื่อกำหนดหมายเลข board
4. Connector ขนาด 40 pin ถูกใช้เชื่อมต่อระหว่างอุปกรณ์ฮาร์ดแวร์ชิ้นนี้และอุปกรณ์ที่ต้องการ ประกอบด้วย data bus, address bus, write (WrB), read (RdB), และ ready (RdyB)



รูปที่ 3.8 Block diagram

### 3.6.3 ชื่อและตำแหน่งขาสัญญาณ



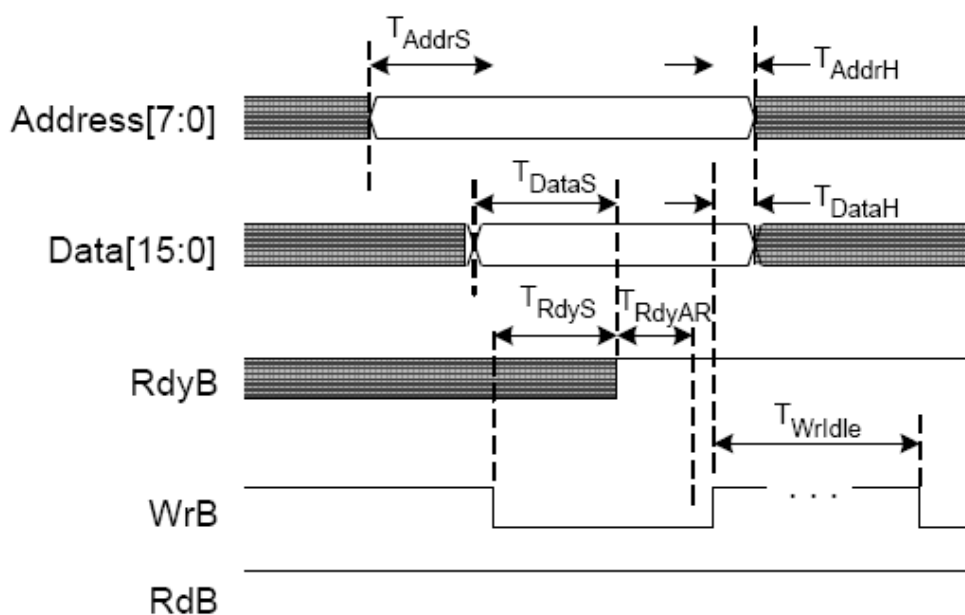
รูปที่ 3.9 ตำแหน่งของขาสัญญาณที่ connector

ตารางที่ 3.1 แรงดันสูงสุดที่แต่ละ pin connector

| ชื่อสัญญาณ   | คำอธิบาย            | ประเภทของสัญญาณ | แรงดันสูงสุด    |
|--------------|---------------------|-----------------|-----------------|
| VCC          | Device Power Supply | OUT             | 3.3             |
| GND          | Device Ground       | OUT             | 0               |
| D0 – D15     | Data bus            | OUT             | 2.0 - 0.5 / 3.3 |
| ADR0 – ADR15 | Address bus         | OUT             | 2.0 - 0.5 / 3.3 |
| WrB          | Write signal        | OUT             | 3.3             |
| RdB          | Read signal         | OUT             | 3.3             |
| RdyB         | Ready signal        | IN              | 2.0 – 0.5       |

### 3.6.4 Timing diagram ของการเขียน

- กรณีที่สัญญาณ RdyB มีค่าเท่ากับ high ในช่วง Ready Assertion to Release Time



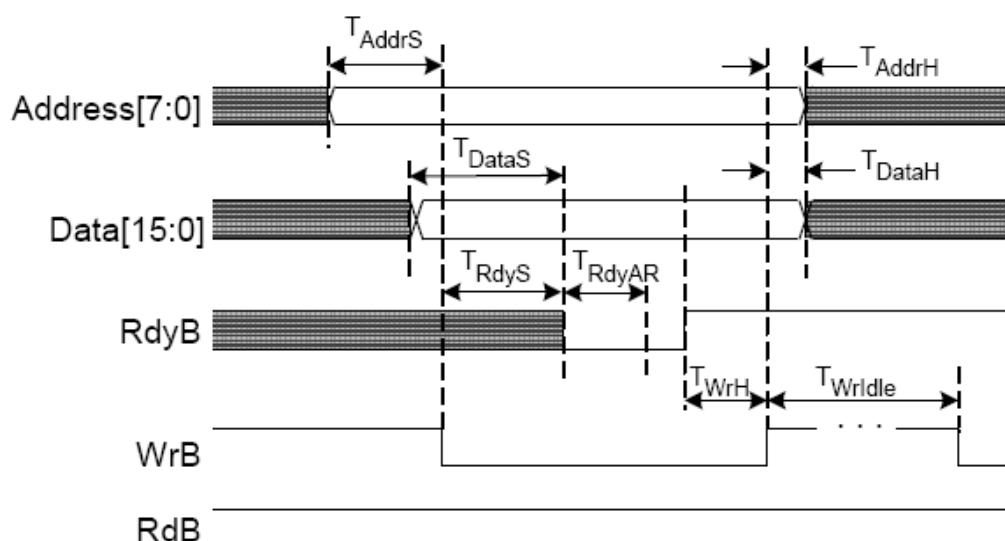
รูปที่ 3.10 รูปสัญญาณต่าง ๆ ในกรณีที่สัญญาณ RdyB มีค่าเท่ากับ high ในช่วง  $T_{RdyAR}$

จากรูปที่ 3.10 คือ สัญญาณต่าง ๆ เมื่อเขียนข้อมูล 1 word ตารางด้านล่างคือค่า setup time, hold time ของสัญญาณต่าง ๆ ที่จำเป็น

ตารางที่ 3.2 ช่วงการเขียนข้อมูล 1 word เมื่อสัญญาณ RdyB มีค่าเท่ากับ high ในช่วง  $T_{RdyAR}$

| Symbol       | Parameter                            | Min. | Max. | Unit |
|--------------|--------------------------------------|------|------|------|
| $T_{AddrS}$  | Address Setup Time                   | 3.5  |      | us   |
| $T_{AddrH}$  | Address Hold Time                    | 2.6  |      | us   |
| $T_{DataS}$  | Data Setup Time                      |      | 310  | ns   |
| $T_{DataH}$  | Data Hold Time                       | 25   |      | ns   |
| $T_{RdyS}$   | Ready Setup Time                     |      | 240  | ns   |
| $T_{RdyAR}$  | Ready Assertion to Release Time      | 15   |      | ns   |
| $T_{WrIdle}$ | Idle Time before continue next write | 6    |      | us   |

2. กรณีที่สัญญาณ RdyB มีค่าเท่ากับ low ในช่วง Ready Assertion to Release Time



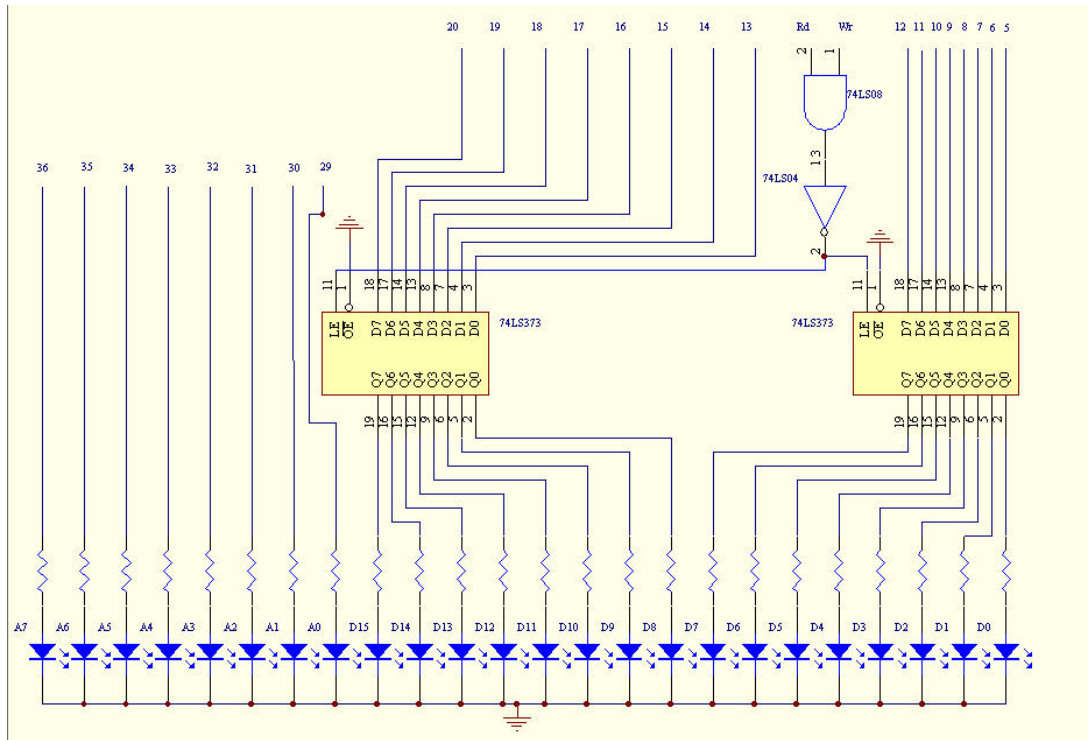
รูปที่ 3.11 รูปสัญญาณต่าง ๆ ในกรณีที่สัญญาณ RdyB มีค่าเท่ากับ low ในช่วง  $T_{RdyAR}$

จากรูปที่ 3.11 คือสัญญาณต่าง ๆ เมื่อเขียนข้อมูล 1 word ตารางด้านล่างคือค่า setup time, hold time ของสัญญาณต่าง ๆ ที่จำเป็น

ตารางที่ 3.3 ช่วงการเขียนข้อมูล 1 word เมื่อสัญญาณ RdyB มีค่าเท่ากับ low ในช่วง  $T_{RdyAR}$

| Symbol       | Parameter                            | Min. | Max. | Unit |
|--------------|--------------------------------------|------|------|------|
| $T_{AddrS}$  | Address Setup Time                   | 3.5  |      | us   |
| $T_{AddrH}$  | Address Hold Time                    | 2.6  |      | us   |
| $T_{DataS}$  | Data Setup Time                      |      | 310  | ns   |
| $T_{DataH}$  | Data Hold Time                       | 25   |      | ns   |
| $T_{RdyS}$   | Ready Setup Time                     |      | 240  | ns   |
| $T_{RdyAR}$  | Ready Assertion to Release Time      | 15   |      | ns   |
| $T_{WrH}$    | Write Hold Time                      | 250  |      | ns   |
| $T_{WrIdle}$ | Idle Time before continue next write | 6    |      | us   |

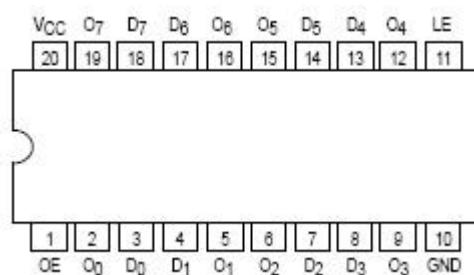
### 3.7 การออกแบบอุปกรณ์ที่ใช้ในการทดลอง



รูปที่ 3.12 แสดงวงจรบอร์ดแสดงผล

จากรูปที่ 3.12 จะแสดงวงจรของบอร์ดที่ใช้ในการแสดงผลซึ่งในวงจรนี้เราจะใช้ไอซีเบอร์ 74LS373 ในการที่จะเอาข้อมูลแสดงผลเนื่องจากถ้าเราไม่นำเอาไอซีเบอร์ 74LS373 มาทำการค้างสถานะไว้ข้อมูลที่ออกมาจะกระพริบแล้วก็หายไป เพราะข้อมูลที่ออกมาจากโมดูล USB นั้นไม่ได้ทำการค้างสถานะเอาไว้ ซึ่งทำให้เราไม่สามารถที่จะรู้ได้ว่า ข้อมูลที่ออกมานั้น ออกมาตรงกับที่เราไปหรือไม่ ซึ่งในการเอาไอซีเบอร์ 74LS373 มาใช้ในการค้างสถานะของข้อมูลเอาไว้ แต่ในส่วนของเราเราไม่จำเป็นต้องทำการค้างสถานะเพราะที่ตัวโมดูลจะมีการค้างสถานะไว้ให้อยู่แล้ว ซึ่งที่ขา OE เราจะป้อน 0 ให้นันเพื่อให้นันทำงานตลอด ส่วนขา enable เราจะใช้ AND GATE และ NOT GATE ในการควบคุมให้นันค้างสถานะเอาไว้

## ไอซีเบอร์ 74LS373



รูปที่ 3.13 แสดงตัวถังของไอซีเบอร์ 74LS373

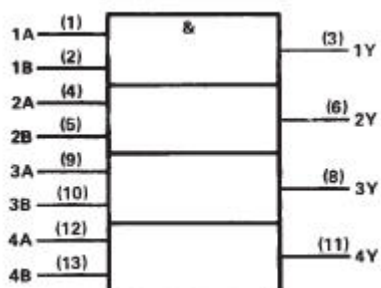
**LS373**

| D <sub>n</sub> | LE | OE | Q <sub>n</sub> |
|----------------|----|----|----------------|
| H              | H  | L  | H              |
| L              | H  | L  | L              |
| X              | L  | L  | Q <sub>0</sub> |
| X              | X  | H  | Z*             |

H = HIGH Voltage Level  
 L = LOW Voltage Level  
 X = Immaterial  
 Z = High Impedance

รูปที่ 3.14 แสดงตารางการทำงานของ ไอซีเบอร์ 74LS373

## ไอซีเบอร์ 74LS08



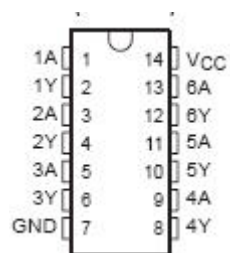
รูปที่ 3.15 แสดงโครงสร้างภายในไอซีเบอร์ 74LS08

FUNCTION TABLE (each gate)

| INPUTS |   | OUTPUT |
|--------|---|--------|
| A      | B | Y      |
| H      | H | H      |
| L      | X | L      |
| X      | L | L      |

รูปที่ 3.16 แสดงตารางการทำงานของไอซีเบอร์ 74LS08

## ไอซีเบอร์ 74LS04



รูปที่ 3.17 แสดงตัวถังเบอร์ไอซีเบอร์ 74LS04

FUNCTION TABLE  
(each inverter)

| INPUT<br>A | OUTPUT<br>Y |
|------------|-------------|
| H          | L           |
| L          | H           |

รูปที่ 3.18 แสดงตารางการทำงานไอซีเบอร์ 74LS04